Figure 7.3 The architecture of our example application, after abstracting out all details relative to the generation of the summary into an ad-hoc component, the “sentiment generator”, that we can imagine hosted on the web server, and is called synchronously from the web app.

It looks much cleaner now, right? Remember, we are hiding all complexity into the “sentiment generator” because we want to focus on the caching mechanism, and we are not interested in the details of how this sentiment is computed. Nonetheless, this doesn’t mean that you can only use caching in-memory or for standalone applications: on the contrary, web applications are probably one of the best places to add some caching layers – the detailed example from where we started should have made this clear enough.

Moreover, it is important to remember that simplifying complex things by abstracting low-level details is a paramount technique in algorithm analysis: for instance, we often assume that some containers would store integers instead of worrying about every possible type of entry that could be stored, and even the RAM model itself (which we introduced in appendix B), is a simplification to hide details of the myriad of different configurations that real computers can have.

7.2.1 Description and API

As always, let’s draft an API and a contract that our data structure should adhere to: